AP HOGESCHOOL ANTWERPEN

Web Technology

Syllabus 2026-2027

Kristof Michiels
AP Hogeschool Antwerpen
versie van 18 september 2026

Inleiding

Dit is de gloednieuwe syllabus voor het vak Web Technology. Ze vergezelt de slides van elk hoorcollege en de labo's waarin je elke week de leerstof inoefent.

In zo'n tekst kruipt veel werk. Telkens een nieuw hoofdstuk klaar is, wordt hier een nieuwe versie van de syllabus gepost. Er wordt naar gestreefd dat de tekst tegen het einde van het semester is uitgegroeid tot een volwaardig handboek: cursustekst bij de lessen, zelfstandig doorwerkbaar studiemateriaal, en een naslagwerk dat ook in de volgende opleidingsjaren bruikbaar blijft. Het niveau ligt bewust wat hoger dan wat je van een eerste semester verwacht; wat dieper gaat dan de les, staat in de kaders en is als zodanig gemarkeerd.

Bij de ontwikkeling van deze syllabus werd generatieve AI gebruikt als ondersteunend hulpmiddel voor onder meer redactie, structurering en uitwerking. Alle inhoud werd door de auteur gecontroleerd, bewerkt en gevalideerd. De auteur draagt de volledige inhoudelijke verantwoordelijkheid.

Inhoud

Hoofdstuk 1 - Het web als systeem

In een notendop

Wat je na dit hoofdstuk kan

1.1 Waarom dit vak bestaat

1.2 Het web als systeem

1.3 Je werkinstrumenten

1.4 HTML: elementen, tags en attributen

1.5 Het skelet van elk HTML-document

1.6 De DOM: jouw document is een boomstructuur

1.7 Tekst structureren

1.8 Semantiek: betekenis geven aan delen van de pagina

1.9 Links: de essentie van het web

1.10 Afbeeldingen

1.11 Meer dan tekst en beeld: video, audio en iframe

1.12 Namen voor later: id en class

1.13 Je pagina online

Onder de motorkap: van adresbalk tot pixels

Veelgemaakte fouten

Met AI aan je zij: gegenereerde HTML lezen

Oefeningen

Verder lezen

Hoofdstuk 1 - Het web als systeem

In een notendop

Het web is een gesprek tussen een client en een server, en een webpagina is een boomstructuur van elementen die betekenis geven aan inhoud. Dit hoofdstuk legt uit hoe dat gesprek verloopt (URL, HTTP, bestanden op een server), welke drie instrumenten je dit semester dagelijks gebruikt (VS Code, de browser met zijn DevTools, GitHub) en hoe je correct en betekenisvol HTML schrijft: het skelet, de boom, tekst, structuur, links en media. Aan het einde staat je eerste pagina online, en je weet waarom een fout in HTML niet luid faalt maar stil.

Wat je na dit hoofdstuk kan

Aan het einde van deze eerste week staat er een webpagina door jou gemaakt op het internet. Een echte pagina, met een echt adres, die van waar ook ter wereld geraadpleegd kan worden. Ze zal er nog eenvoudig uitzien - voor vormgeving is het wachten tot hoofdstuk 2 - maar ze bevat alles wat elke webpagina ter wereld bevat, van de site van je bank tot de startpagina van Wikipedia: een gestructureerde **boom van HTML-elementen** die betekenis geven aan informatie.

Onderweg leer je hoe het **web als systeem** in elkaar zit: wat er gebeurt tussen het intikken van een adres en het verschijnen van een pagina, hoe je met de drie kerninstrumenten van dit vak werkt (Visual Studio Code, de browser met zijn DevTools, en GitHub), en vooral hoe je **correct en betekenisvol HTML** schrijft. Dat laatste lijkt de eenvoudigste vaardigheid van dit vak. Het is ook de vaardigheid waarop alles rust wat volgt; wie ze slordig aanleert, betaalt daar in elk volgend hoofdstuk een prijs voor.

Het hoofdstuk heeft drie delen. Eerst het systeem en je gereedschap (1.1 tot 1.3): waarom dit vak bestaat, hoe client, server en browser samenwerken, en welke drie programma's je installeert. Dan de taal (1.4 tot 1.12): elementen, het skelet, de boom, en element per element wat je nodig hebt voor tekst, structuur, links en media. Ten slotte de laatste stap (1.13): je pagina online.

1.1 Waarom dit vak bestaat

Begin bij de vraag die je je waarschijnlijk al stelde: waarom zou je in 2026 nog leren om HTML, CSS en JavaScript met de hand te schrijven, als AI dat in seconden voor je doet?

Het antwoord: omdat AI dat inderdaad in seconden voor je doet. Elke opdracht in de cursus en de labo's kan een taalmodel oplossen. Als je dit vak benadert als een verzameling opdrachten die "af" moeten, dan kan je elk labo uitbesteden aan een chatbot en zal je elke week het gevoel hebben dat

je goed bezig bent. Tot de eerste toets die je zonder AI maakt. Dit overkomt elk jaar studenten. Laat het niet jouw ervaring zijn.

Waarom leggen we de lat dan net hoger in plaats van lager? Omdat er iets fundamenteels veranderd is aan wat een webdeveloper doet. Code produceren is goedkoop geworden. Code begrijpen, beoordelen en corrigeren is dat niet - en dat is precies waar het beroep naartoe beweegt. Een developer typt vandaag zelden nog een layout from scratch, maar leest voortdurend code die een model genereerde, en moet dan kunnen zien: dit klopt, dit klopt bijna, en dit is subtiel fout op een manier die over drie weken een probleem wordt. Dat zien vereist een model in je hoofd van hoe het web werkt. Wie dat model niet heeft, kan alleen maar hopen dat de chatbot gelijk heeft.

Dit vak bouwt dat model, laag per laag: eerst de boom van elementen (dit hoofdstuk), dan het "geordende conflict" dat CSS heet, dan het boxmodel, de layouts, en met JavaScript uiteindelijk de code die op gebruikers reageert. Handmatig kunnen bouwen is in dit vak geen nostalgisch ritueel; het is het bewijs dat het model in je hoofd zit. Daarom werken we bovendien *vanilla-only*: puur HTML, CSS en JavaScript, zonder frameworks of libraries. Het platform zelf kan vandaag meer dan ooit, en wie het platform kent, ziet later moeiteloos dat elke framework-laag daarbovenop dun is. Frameworks krijg je volgend jaar, in het vak Web Frameworks. Eerst dus het fundament.

Hoe we in dit vak met AI werken

We gaan niet doen alsof AI niet bestaat; we maken er expliciete afspraken over. Elke oefening in deze syllabus en elk labo draagt één van drie labels:

- **zonder AI**: je gebruikt geen AI. Dit label staat op de vaardigheidstoetsen en op oefeningen die daarop voorbereiden. Tijdens toetsen en examens is dit een feit: daar is geen AI beschikbaar
- **zonder AI aanbevolen**: je mag AI gebruiken, maar je traint dan minder jezelf. Je haalt er meer uit door het niet te doen. Dit label staat op de meeste drills
- **met AI**: je gebruikt AI en dat is het punt van de oefening. Je genereert, beoordeelt, verbetert en verantwoordt. Wat we oefenen, is jouw kritiek op wat een model maakte

Nog één afspraak: vanaf labo 2 wordt alles wat je in je repository zet automatisch nagekeken door onze feedback-tooling. In de loop van de week die daarop volgt hoor je waar je staat, met verwijzingen naar de sectie in deze syllabus waar de oplossing wordt uitgelegd. Die feedback is er om jou te helpen en telt niet mee voor je punten; je punten komen uit de toetsen en het examen. We zien als docenten wel wie meedoet en wie stilvalt - niet om te controleren, maar om je op tijd te kunnen helpen in plaats van pas in januari.

1.2 Het web als systeem

Voordat we HTML schrijven, moet je begrijpen wat er eigenlijk gebeurt als je een website bezoekt. Nog geen expertkennis - daar hebben we andere vakken en semesters voor - maar goed genoeg om te weten waar jouw werk straks past in het geheel.

Het web bestaat uit twee soorten spelers. Een **server is een computer die bestanden bewaart en op verzoek uitdeelt**; hij staat ergens in een datacenter (maar het kan ook jouw computer zijn) en doet niets anders dan wachten op verzoeken. Een **client is het programma dat die verzoeken doet**: in ons geval de browser op jouw laptop of smartphone. Het hele web is een eindeloze herhaling van hetzelfde gesprek: de client vraagt een bestand op, de server stuurt het terug, de client toont het resultaat.



Figuur 1.1 - De client (je browser) vraagt met een URL een bestand aan de server; de server antwoordt met een statuscode en het bestand, en de browser toont het.

Het adres: de URL

Het adres waarmee de client vraagt, ken je al jaren: de **URL (Uniform Resource Locator)**. Een URL heeft een vaste anatomie, en je kan hem lezen zoals je een adres op een envelop leest:

```
https://www.ap.be/opleidingen/toegepaste-informatica
└──┘   └─────────┘└──────────────────────────────────┘
schema     host                  pad
```

Het **schema** (`https`) zegt welke taal client en server spreken. De **host** (`www.ap.be`) identificeert de server: een naam die via het DNS-systeem wordt vertaald naar het ip-adres van een machine. Het **pad** wijst naar één specifiek bestand of één specifieke pagina op die server. Onthoud vooral dat laatste: een URL wijst altijd ergens naartoe. Straks, wanneer je links en afbeeldingen schrijft, is elke `href` en elke `src` gewoon zo'n adres - lang of kort geschreven, maar altijd een wegbeschrijving naar een bestand.

Het gesprek: HTTP en HTTPS

De taal van het gesprek zelf heet **HTTP** (HyperText Transfer Protocol), en voor dit vak volstaat het in vijf zinnen. Eén: de browser stuurt een verzoek naar de server, met daarin de URL en wat context. Twee: de server zoekt het gevraagde bestand op. Drie: de server stuurt een antwoord terug, met een statuscode (200 betekent "hier is het", 404 betekent "bestaat niet") en het bestand zelf. Vier: de browser leest het bestand en ontdekt daarin verwijzingen naar méér bestanden - stylesheets, afbeeldingen, scripts - en herhaalt het gesprek voor elk daarvan. Vijf: uit al die bestanden samen bouwt de browser de pagina die jij ziet. Meer HTTP heb je dit semester niet nodig; in het tweede semester ga je zelf zulke gesprekken voeren vanuit code.

De **S** in `https` staat voor **secure**: hetzelfde gesprek, maar versleuteld. Zonder die S reist alles wat client en server elkaar sturen in leesbare tekst over het netwerk, en iedereen die ergens onderweg zit - de wifi van de koffiebar, je internetprovider, een router in een datacenter - kan meelezen én de inhoud onderweg veranderen. Met HTTPS gebeuren twee dingen. De verbinding wordt versleuteld, zodat meelezen zinloos wordt. En de server bewijst met een **certificaat** wie hij is, zodat je zeker weet dat `www.ap.be` echt de server van AP is en niet een kopie die iemand ertussen zette. Daarom toont je browser bij `http://` de waarschuwing *Niet beveiligd*, en daarom is HTTPS vandaag de norm en niet de uitzondering: ook voor een pagina zonder wachtwoorden of betalingen, want de bezoeker moet erop kunnen vertrouwen dat hij jouw pagina ziet en niet een aangepaste versie. Voor jou verandert dat weinig: GitHub Pages serveert je site automatisch via HTTPS, en de enige regel die je moet onthouden is dat alle adressen die je in je pagina's schrijft met `https://` beginnen. Een `http://`-afbeelding op een `https://`-pagina blokkeert de browser gewoon.

De bestanden op de server

Waar komen die bestanden op de server vandaan? Een website is - op het niveau waarop we dit semester werken - letterlijk een map met bestanden: HTML-bestanden voor de structuur en inhoud, CSS-bestanden voor de vormgeving, afbeeldingen, en straks JavaScript-bestanden voor de interactie. "Een site online zetten" betekent: die map op een server plaatsen. Dat is precies wat jij deze week doet, want een pagina staat online of ze bestaat niet. Een oefening die alleen op jouw laptop leeft, kan niemand bezoeken.

Eén bestandsnaam heeft een speciale rol: `index.html`. Wanneer een URL naar een map wijst in plaats van naar een bestand (zoals `https://www.ap.be/`), zoekt de server in die map naar `index.html` en stuurt dat terug. Daarom heet de startpagina van vrijwel elke website zo, en daarom heten de startbestanden van al jouw oefeningen zo.

De drie bouwstenen

Elke webpagina, hoe complex ook, is opgebouwd uit maximaal drie talen, elk met een strikte taakverdeling:

- **HTML** (HyperText Markup Language) beschrijft de *structuur en betekenis* van de inhoud: dit is een titel, dit is een paragraaf, dit is een lijst, dit is navigatie
- **CSS** (Cascading Style Sheets) beschrijft de *vormgeving*: lettertypes, kleuren, witruimte, layout, tot en met animatie
- **JavaScript** beschrijft het *gedrag*: wat er gebeurt als de gebruiker klikt, typt, scrolt

Die taakverdeling is een ontwerpprincipe dat je vanaf vandaag hanteert. HTML gaat nooit over vorm. Je kiest een element nooit omdat het er groot of vet uitziet, maar omdat het de juiste betekenis draagt; hoe het eruitziet, beslis je later in CSS, en dat kan er compleet anders uitzien dan wat de browser standaard toont.

Een beknopte geschiedenis van het web

Het (wereldwijde) web - ook www genoemd - is geen ontwerp van één bedrijf; het groeide in ruim dertig jaar uit een handvol beslissingen die vandaag nog altijd gelden. Twee begrippen worden daarbij vaak door elkaar gehaald. Het **internet** is het netwerk: de kabels, routers en afspraken (TCP/IP) waarmee computers wereldwijd pakketjes data naar elkaar sturen. Het bestaat sinds de jaren zeventig en regelde lang vóór het web al e-mail en bestandsoverdracht. Het **web** is één toepassing op dat netwerk: documenten met links ertussen, opgevraagd via HTTP en geschreven in HTML. Het is "uitgevonden" in 1989 door Tim Berners-Lee, een informaticus bij het CERN in Genève, die een manier zocht om onderzoekers documenten te laten delen. Zijn drie afspraken - de URL als adres, HTTP als gesprek, HTML als documenttaal - zijn precies de drie die je zonet leerde, en de beslissing van het CERN in 1993 om die technologie gratis en zonder patent vrij te geven, is de reden dat het web van iedereen is en niet van een bedrijf.

De tijdlijn hieronder bevat de gebeurtenissen die je als IT-er moet kunnen plaatsen, met telkens waarom ze voor jou nog relevant zijn.

JAAR	GEBEURTENIS	WAAROM HET VOOR JOU BELANGRIJK IS
1969	ARPANET, de voorloper van het internet, verbindt de eerste computers	het netwerk is ouder dan het web
1983	TCP/IP wordt de standaard van het internet; het DNS wordt ingevoerd	namen zoals `www.ap.be` bestaan sinds dan
1989	Tim Berners-Lee stelt bij het CERN het World Wide Web voor	URL, HTTP en HTML worden in één ontwerp geboren
1991	de eerste website gaat online; het web verlaat het CERN	een website is nog altijd wat het toen was: een map met documenten
1993	het CERN geeft het web vrij; de browser Mosaic toont afbeeldingen in de pagina	het web wordt van iedereen, en het wordt visueel
1994	het W3C wordt opgericht; Netscape Navigator verschijnt	er is een orgaan dat standaarden schrijft, en een browser die ze al dan niet volgt
1995	JavaScript wordt in tien dagen gebouwd bij Netscape; HTML 2.0 is de eerste officiële HTML-standaard	de derde bouwsteen bestaat, met haar eigenaardigheden (hoofdstuk 10)
1996	CSS1 wordt een aanbeveling van het W3C	vorm wordt losgekoppeld van structuur: de taakverdeling van 1.2
1998	DOM Level 1; Google wordt opgericht	de boom van 1.6 krijgt een standaard-API; zoekmachines worden een lezer van je pagina
1997-2001	de browseroorlog tussen Netscape en Internet Explorer	jarenlang moest elke site per browser anders geschreven worden; de reden dat standaarden uiterst belangrijk zijn
2004	Firefox 1.0; Gmail toont wat een pagina kan zonder te herladen (Ajax); de WHATWG wordt opgericht	de browser wordt een platform voor toepassingen
2007	de iPhone	vanaf nu is hét scherm een veelheid aan schermen: hoofdstuk 6
2008	Google Chrome verschijnt, met een snelle JavaScript-engine	JavaScript wordt een serieuze taal; de DevTools van 1.3 zijn Chromes erfenis
2014	HTML5 wordt de officiële standaard	de semantische elementen van 1.8 (header, nav, main, article) en video en audio (1.11)
2015	ES2015 (ES6) vernieuwt JavaScript grondig; HTTP/2	het moderne JavaScript-idioom van deze syllabus
2017	CSS Grid werkt in alle grote browsers	layout krijgt eindelijk een speciaal daarvoor ontwikkelde CSS-techniek (hoofdstuk 4)

JAAR	GEBEURTENIS	WAAROM HET VOOR JOU BELANGRIJK IS
2019	W3C en WHATWG spreken af: HTML is één "living standard"	versienummers zijn voorbij; er is alleen nog "HTML"
2022-2023	container queries, `:has()` en CSS nesting; ChatGPT verschijnt; Baseline wordt ingevoerd	het platform kan meer dan ooit, code genereren wordt triviaal, en "kan ik dit al gebruiken?" krijgt een helder antwoord
2025	de European Accessibility Act treedt in werking	toegankelijkheid is een wettelijke eis (1.8)

Twee lessen uit die tijdlijn. Ten eerste: de afspraken van 1989 zijn nooit vervangen, alleen uitgebreid. Wie URL, HTTP en HTML begrijpt, begrijpt het web van vandaag. Ten tweede: het web is geen product maar een standaard, geschreven door organisaties waarin browsermakers samen beslissen (het **W3C** voor CSS en toegankelijkheid, de **WHATWG** voor HTML en de DOM, **Ecma** voor JavaScript). Daarom kan je op MDN per feature opzoeken of ze al in alle browsers werkt, en daarom bestaat de code die je dit semester schrijft over tien jaar nog.

1.3 Je werkinstrumenten

In dit vak gebruik je drie instrumenten, en je moet ze alle drie goed onder de knie krijgen.

Visual Studio Code is je editor: het programma waarin je code schrijft. Installeer daarnaast meteen de extensie *Live Server*: die toont je pagina in de browser en ververst automatisch bij elke wijziging die je bewaart. Code voor het web is tekst; een editor is gebouwd om tekst exact te behandelen. Schrijf nooit code in een tekstverwerker - die "verbetert" je aanhalingstekens en breekt daarmee je bestanden.

De browser (met DevTools). We gebruiken in de lessen Chrome, en kijken er je werk mee na, maar Firefox of Edge werken net zo goed. Belangrijker dan de browserkeuze is één toetscombinatie die je vandaag nog uit je hoofd leert: die voor de **DevTools** (rechtsklik op een pagina en kies *Inspect*, of F12 op Windows en Cmd-Option-I op Mac). De DevTools zijn het instrumentenpaneel van de browser, en het tabblad *Elements* toont je iets essentieels: niet de tekst die de developer schreef, maar de boom die de browser ervan bouwde. Vanaf vandaag is dit je eerste reflex bij elke vraag over een pagina - de jouwe of die van iemand anders. Open de inspector, kijk naar de boom, en je ziet wat de browser ziet. In dit vak leren we de DevTools niet "erbij"; het is een kerninstrument dat in elk hoofdstuk terugkomt.

GitHub is de plek waar jouw werk leeft en vanwaar het gepubliceerd wordt. Het concept in drie zinnen. Op je computer heb je een map met je bestanden; op GitHub staat een kopie van die map, je **repository** (kortweg: repo). Telkens je een betekenisvolle stap hebt gezet, breng je beide in overeenstemming en schrijf je in één zin wat je deed. GitHub bewaart daarbij de volledige geschiedenis, en publiceert de inhoud via **GitHub Pages** automatisch als website op een echt adres.

Meer Git-theorie dan dit heb je dit semester niet nodig - geen terminal, geen commando's, geen branches. In labo 1 zet je je eerste pagina's online zoals het eenvoudigst kan: je maakt in de browser een repository `webtech` aan, sleept je map erin en zet Pages aan. Vanaf labo 2 gaat het rechtstreeks vanuit VS Code, met één klik (*Commit & Sync*); de volledige stappenkaart met schermafbeeldingen daarvoor vind je in **appendix A**, en vanaf dan is publiceren een routine van tien seconden. Waarom we dit vanaf week 1 doen en niet "later, als het er echt toe doet": ten eerste omdat een gepubliceerde site het vak zichtbaar en echt maakt, ten tweede omdat je repository vanaf labo 2 ook de manier is waarop je labowerk wordt ingeleverd en automatisch van feedback voorzien, en ten derde omdat werken via repositories dé samenwerkingsvorm is van moderne development - ook, en steeds meer, de samenwerking tussen developers en AI-agents.

Andere manieren om te publiceren

GitHub Pages is niet de enige weg. Elke dienst die een map met bestanden op een server zet, doet hetzelfde werk als in 1.2 beschreven, en het loont om er minstens één andere te kennen. **Netlify** is de snelste: ga naar app.netlify.com, kies *Drop*, sleep je map in het venster, en enkele seconden later staat ze online op een adres als `https://willekeurige-naam.netlify.app`. Geen repository, geen instellingen; de gratis versie volstaat voor een studentensite. Netlify kan ook aan een GitHub-repository gekoppeld worden, zodat het net als Pages bij elke wijziging opnieuw publiceert. Verwante diensten zijn Cloudflare Pages en Vercel, en de klassieke variant is een hostingpakket waarbij je de map met een FTP-programma naar de server kopieert - de manier waarop het web twintig jaar werkte en waarop veel kleine sites nog altijd draaien. Voor dit vak kiezen we GitHub Pages, omdat publiceren dan samenvalt met het bewaren van je geschiedenis en met inleveren. Maar onthoud het principe: een site is een map, en publiceren is die map ergens neerzetten waar een server ze kan uitdelen.

1.4 HTML: elementen, tags en attributen

Tijd voor de taal zelf. Kijk naar deze vier regels; ze zijn stuk voor stuk HTML-elementen:

```
<h1>Ik ben een hoofding</h1>
<p>Ik ben een paragraaf met daarin <em>benadrukte</em> tekst.</p>
<a href="https://www.ap.be">Ik ben een link naar de AP-site</a>
<img src="logo.png" alt="Het logo van de opleiding">
```

Een **element** is de combinatie van tags en inhoud. De meeste elementen bestaan uit een **openingstag** (`<p>`), inhoud, en een **sluittag** (`</p>` - let op de slash). De tags geven betekenis aan de inhoud ertussen: dezelfde woorden worden een hoofding, een paragraaf of een link, puur door de tags die je kiest. Sommige elementen hebben geen inhoud en dus ook geen sluittag; `<img>` is er zo één - er bestaat geen "tekst binnen een afbeelding", dus het element is in zichzelf compleet. We noemen dat een **void element**.

In de openingstag kunnen **attributen** staan: naam-waardeparen die extra informatie aan het element koppelen. Ze hebben altijd dezelfde vorm - naam, gelijkheidsteken, waarde tussen aanhalingstekens - en staan gescheiden door spaties:

```
<a href="https://www.ap.be">...</a>
<img src="logo.png" alt="Het logo van de opleiding">
```

Sommige attributen zijn verplicht (`href` bij een link, `src` en `alt` bij een afbeelding), de meeste zijn optioneel. Welke attributen een element kent, zoek je op; niemand kent ze allemaal uit het hoofd, en de referentie die we consequent gebruiken is **MDN Web Docs** (developer.mozilla.org) - de documentatie van het webplatform zelf.

Elementen kunnen ten slotte in elkaar **genest** worden: de paragraaf hierboven bevat een `em`-element. Nesting is geen detail; het is het hele organisatieprincipe van HTML, en het brengt ons bij de belangrijkste denkstap van dit hoofdstuk. Maar eerst het vaste vertrekpunt van elk document.

1.5 Het skelet van elk HTML-document

Elk HTML-bestand dat je dit semester schrijft, begint met exact dit skelet:

```
<!doctype html>
<html lang="nl">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Titel van deze pagina</title>
  </head>
  <body>
    <!-- your page content goes here -->
  </body>
</html>
```

Regel voor regel. De **doctype** vertelt de browser: dit is modern HTML, behandel het volgens de huidige standaard. Het **html-element** is het root-element dat al de rest omvat; het `lang`-attribuut vertelt aan browsers, zoekmachines en schermlezers in welke taal de pagina geschreven is. Dat lijkt een detail, maar een schermlezer die Nederlands voorleest met een Engelse stem produceert brij - vul `lang` dus altijd correct in.

Binnen het root-element leven precies twee kinderen. De **head** bevat informatie *over* de pagina, die niet op de pagina zelf verschijnt: de karaktercodering (`utf-8`, zodat accenten en emoji correct werken), de viewport-instelling (die vertelt mobiele browsers dat je pagina zich aan hun schermbreedte aanpast - het volledige verhaal volgt in hoofdstuk 6, maar hij staat er vanaf dag één), en de **title**, de tekst die in het browsertabblad en in zoekresultaten verschijnt. De **body** bevat alles wat de bezoeker wél ziet: daarin schrijf jij vanaf nu je pagina's.

De regel `` is **commentaar**: tekst voor menselijke lezers die de browser volledig negeert. Je zet er notities mee in je code, of je schakelt er tijdelijk een stuk opmaak mee uit. Gebruik het om te verhelderen wat niet vanzelf spreekt - voor de anderen die jouw code lezen, en voor jezelf over drie weken.

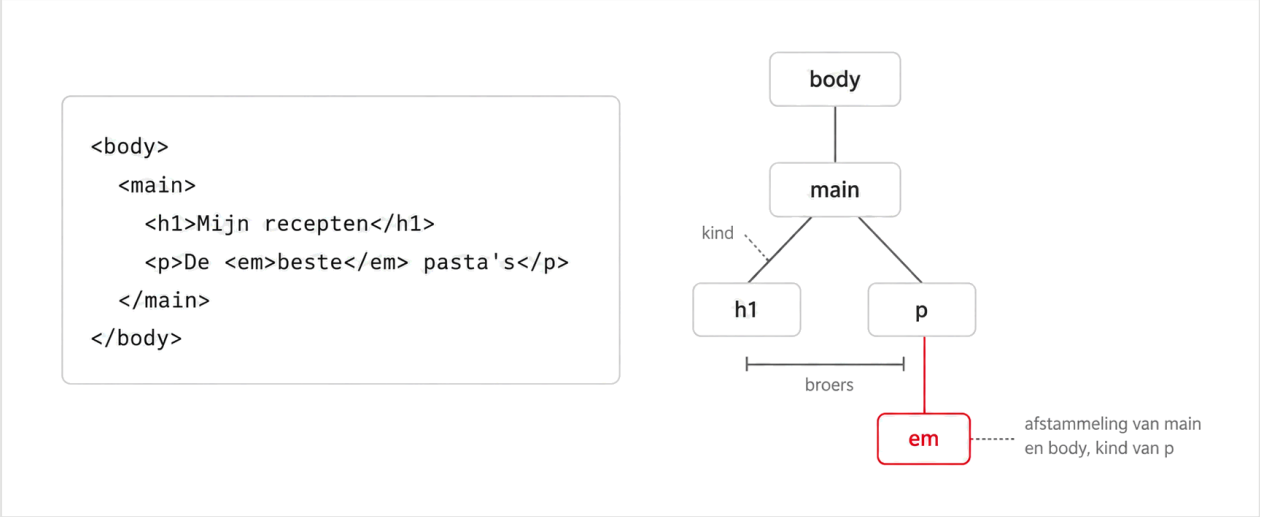
Merk ten slotte de insprongen op: elk genest element springt één niveau in. De browser is daar blind voor - HTML is ongevoelig voor extra witruimte - maar mensen niet. Insprongen zijn hoe jij, je medestudenten, je docenten en straks je collega's de structuur van een document in één oogopslag lezen. Netjes inspringen is in dit vak geen stilistische voorkeur maar een vereiste, om dezelfde reden als leesbaar handschrift bij een wiskundebewijs: wie de structuur niet toont, maakt fouten die hij zelf niet meer ziet.

1.6 De DOM: jouw document is een boomstructuur

Nu de denkstap die dit hoofdstuk zijn naam geeft. Kijk nog eens naar het skelet, en let op wie in wie genest zit: `title` zit in `head`, `head` zit in `html`. Nesting maakt van een document een hiërarchie, en die hiërarchie heeft een naam: het **Document Object Model**, kortweg de **DOM**.

Wanneer de browser jouw HTML-bestand ontvangt, leest hij de tekst en bouwt er een boomstructuur van. Elk element wordt een knoop in die boom. We gebruiken daarvoor de woordenschat van een stamboom: `head` en `body` zijn de **kinderen** (children) van `html`, dat hun **ouder** (parent) is; `head` en `body` zijn elkaars **broers** (siblings); en alles wat ergens onder een element hangt, hoe diep ook, is een **afstammeling** (descendant) van dat element.

```
<body>
  <main>
    <h1>Mijn recepten</h1>
    <p>De <em>beste</em> pasta's van het studentenleven.</p>
  </main>
</body>
```



Figuur 1.2 - Hetzelfde fragment als code en als boom: `body` heeft één kind `main`, `main` heeft de kinderen `h1` en `p`, en `em` is een kind van `p` en een afstammeling van `main` en `body`.

Teken dit voorbeeld als boom en je krijgt: `body` heeft één kind (`main`); `main` heeft twee kinderen (`h1` en `p`); `p` heeft één elementkind (`em`). De `em` is een afstammeling van `main` én van `body`, maar een kind van alleen `p`.

Waarom is dit het belangrijkste concept van het hoofdstuk? Omdat álles wat volgt in dit vak op deze boom werkt. CSS-regels selecteren knopen in de boom ("elke `em` die ergens in een `li` zit") en stijlen erven langs de takken naar beneden - dat wordt hoofdstuk 2. JavaScript vraagt knopen op, verandert ze, maakt nieuwe knopen en hangt ze in de boom - dat wordt hoofdstuk 11. En de DevTools die je in 1.3 leerde openen, tonen je niet jouw tekstbestand maar precies deze boom, zoals de browser hem gebouwd heeft. Wie in bomen denkt, leest CSS-selectors straks als gewone zinnen. Wie HTML als "tekst met haakjes" blijft zien, leert elke regel apart uit het hoofd.

Een praktisch gevolg om nu al te onthouden: nesting is alles-of-niets. Een element zit volledig binnen zijn ouder of volledig erbuiten; tags mogen elkaar nooit kruisen. `<p><em>fout</p></em>` is geen boom meer en dus geen geldig document - de browser zal er iets van proberen te maken (daarover meer in het kader *Onder de motorkap*), maar wat hij ervan maakt, is niet meer wat jij bedoelde.

1.7 Tekst structureren

Met het skelet en het boommodel op zak kunnen we het binnenste van de body vullen. De elementen in deze sectie zijn de broodelementen van elk tekstdocument op het web.

Paragrafen en hoofdingen

Het **p-element** (paragraaf) is het werkpaard van tekst. Elke alinea lopende tekst hoort in een eigen `<p>`; tekst die "los" in de body staat, zonder omhullend element, noemen we naakte tekst en is een teken van slordige opmaak - elk stuk informatie verdient een element dat zegt wat het is.

Hoofdingen geven je document zijn geleding, in zes niveaus van `<h1>` tot en met `<h6>`. De niveaus vormen een inhoudsopgave, geen maten-en-gewichtentabel: `h1` is de titel van de pagina (één per pagina), `h2`-elementen zijn de hoofdstukken, `h3`-elementen de onderdelen daarvan, enzovoort. Je kiest een niveau dus nooit omdat het "de juiste grootte" heeft - grootte is CSS - maar omdat het de juiste plaats in de inhoudsopgave uitdrukt, en je slaat geen niveaus over. Test jezelf: als je alleen de hoofdingen van je pagina onder elkaar zet, moet er een logische inhoudsopgave staan. Schermlezers doen letterlijk dit - gebruikers ervan springen van hoofding naar hoofding - en zoekmachines lezen je pagina op dezelfde manier.

```
<h1>Mijn kookblog</h1>
<h2>Pasta carbonara voor beginners</h2>
<p>Deze klassieker vraagt maar vijf ingrediënten.</p>
<h3>Ingrediënten</h3>
<p>Spaghetti, guanciale, eieren, pecorino en zwarte peper.</p>
<h2>Over deze blog</h2>
<p>Koken op kot, zonder oven en zonder budget.</p>
```

Lijsten

Alles wat meer dan één keer voorkomt, is een kandidaat-lijst - en op het echte web zijn lijsten overal, veel breder dan boodschappenlijstjes: navigatiebalken, zoekresultaten, productoverzichten en fotogalerijen zijn allemaal lijsten. HTML geeft je er twee soorten. Een **ongeordende lijst** (`<ul>`, unordered list) gebruik je wanneer de volgorde er niet toe doet; een **geordende lijst** (`<ol>`, ordered list) wanneer de volgorde betekenis draagt, zoals bij de stappen van een recept. Binnenin bestaan beide uit **lijstitems** (`<li>`), en dat is meteen een harde regel: een `ul` of `ol` heeft als directe kinderen uitsluitend `li`-elementen.

```
<h3>Bereiding</h3>
<ol>
  <li>Kook de pasta beetgaar.</li>
  <li>Bak de guanciale krokant.</li>
  <li>Meng alles van het vuur af met het eimengsel.</li>
</ol>
```

Binnen een lijstitem mag je vrijwel alles kwijt: tekst, maar ook hoofdingen, paragrafen, afbeeldingen of een nieuwe lijst. Met dat laatste maak je geneste lijsten, en daar toont zich het boommodel opnieuw - let goed op wáár de binnenlijst staat: als kind van een `li`, niet als rechtstreeks kind van de buitenlijst:

```
<ul>
  <li>Pasta's</li>
  <li>
    Sauzen
    <ul>
      <li>Carbonara</li>
      <li>Arrabbiata</li>
    </ul>
  </li>
</ul>
```

De bolletjes en nummers die de browser toont, zijn standaardvormgeving; vanaf hoofdstuk 2 bepaal jij hoe een lijst eruitziet, tot en met lijsten die er helemaal niet meer als "lijstjes" uitzien. De structuur blijft; het uitzicht is vrij. Dat is de taakverdeling van 1.2 in de praktijk.

Accenten binnen tekst: em en strong

Binnen lopende tekst kan je woorden een speciale lading geven. Het **em-element** (emphasis) legt klemtoon zoals je dat sprekend zou doen: "dit was *niet* de bedoeling". Het **strong-element** markeert belang of urgentie: "bewaar dit bestand **voor** je het sluit". De browser toont em standaard cursief en strong standaard vet, maar - je voelt het patroon al - dat is uitzicht en dus aanpasbaar; je kiest deze elementen om wat ze betekenen. Wil je alleen maar dat iets er cursief of vet *uitziet*, zonder betekenislading, dan is dat een taak voor CSS, niet voor markup.

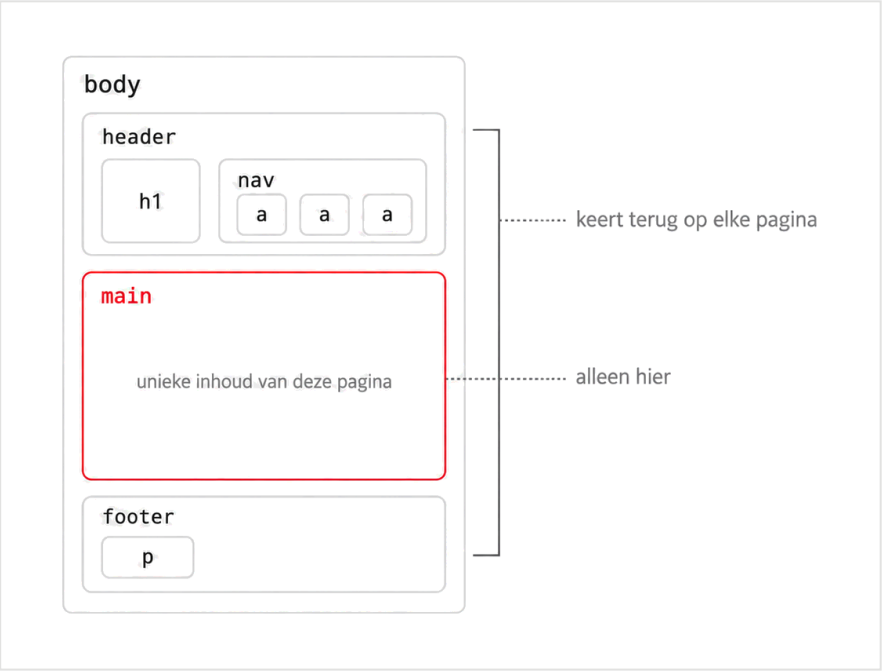
`em` en `strong` gedragen zich anders dan de elementen hierboven: ze leven *binnen* een tekstregel in plaats van er een eigen blok mee te vormen. Dat onderscheid - elementen die als blokken onder elkaar vallen tegenover elementen die in de tekststroom meelopen - zie je op elke pagina, en het krijgt in hoofdstuk 3 een precieze verklaring in het boxmodel. Voor nu volstaat de observatie.

1.8 Semantiek: betekenis geven aan delen van de pagina

De elementen tot nu toe beschrijven kleine eenheden: een titel, een alinea, een lijst. Maar een pagina heeft ook een grofstructuur - een kop, een navigatie, een hoofdinhoud, een voet - en ook daarvoor heeft HTML betekenisdragende elementen. We noemen ze de **semantische verzamelelementen**: containers die andere elementen bundelen én benoemen wat die bundel is.

Het **main-element** trekt een cirkel rond de hoofdinhoud van de pagina: de informatie die deze pagina uniek maakt. Elke pagina heeft er precies één. Het **header-element** bevat de inleidende bovenkant - vaak een logo, de sitenaam en de navigatie - en het **footer-element** de afsluitende onderkant, met bijvoorbeeld auteurs- en contactinformatie. Header en footer horen niet in main thuis: zij bevatten wat op elke pagina van de site terugkeert, main bevat wat alleen hier staat. Het **nav-element** ten slotte markeert een blok navigatielinks - en omdat navigatie een verzameling gelijkaardige links is, herken je er als vanzelf een lijst in:

```html
<body>
  <header>
    <h1>Mijn kookblog</h1>
    <nav>
      <ul>
        <li><a href="index.html">Home</a></li>
        <li><a href="recepten.html">Recepten</a></li>
        <li><a href="over-mij.html">Over mij</a></li>
      </ul>
    </nav>
  </header>
  <main>
    <!-- unique content of this page -->
  </main>
  <footer>
    <p>Gemaakt door Tim Peeters, student Toegepaste Informatica.</p>
  </footer>
</body>
```



Figuur 1.3 - Het grondplan van een pagina: `header` (met `h1` en `nav`) en `footer` keren op elke pagina terug; `main` bevat wat alleen op deze pagina staat.

Dit patroon - header, main, footer als kinderen van body - is het **grondplan** van vrijwel elke webpagina, en het is het startpunt van al je oefeningen vanaf nu. Voor fijnere geleding bestaan nog drie verwanten die je moet kennen: **article** voor een op zichzelf staand stuk inhoud dat ook los van de pagina betekenis zou hebben (een blogpost, een nieuwsbericht, een productkaart), **section** voor een thematisch deel van een pagina dat typisch een eigen hoofding heeft, en **aside** voor inhoud die naast het hoofdverhaal staat (een zijbalk, een "lees ook"-blok). De grenzen tussen die drie zijn soms een kwestie van oordeel; dat oordeel ontwikkel je door te oefenen, en de vuistregels op MDN helpen bij twijfel.

En wat als geen enkel betekenisdragend element past? Dan - en pas dan - grijp je naar de twee neutrale elementen: **div** als betekenisloze blokcontainer en **span** als betekenisloos stukje binnen een tekstregel. Ze zeggen niets over hun inhoud; het zijn haakjes waarmee je elementen kan groeperen om er straks met CSS of JavaScript iets mee te doen. Onthoud de rangorde: eerst een semantisch element zoeken, div en span als laatste redmiddel. In hoofdstuk 2 wordt duidelijk waarom je desondanks véél divs zal zien - en in het kader *Met AI aan je zij* hieronder waarom je er in gegenereerde code nog meer zal zien dan nodig.

Waarom semantiek geen schoonheidsprijs is

Je zou kunnen denken: de bezoeker ziet toch niet welke tags ik gebruikte - waarom zou ik moeite doen als een stapel divs er hetzelfde uitziet? Omdat mensen met ogen op een scherm maar één soort lezer van je pagina zijn, en de andere lezers uitsluitend je markup hebben.

Schermlezers - de voorleessoftware waarmee blinde en slechtziende gebruikers het web gebruiken - navigeren via jouw structuur: van hoofding naar hoofding, rechtstreeks naar main of nav. Een pagina zonder semantiek is voor hen een muur van ongedifferentieerde tekst. Dit is bovendien sinds juni 2025 geen vrijblijvende kwestie meer: de **European Accessibility Act** verplicht digitale toegankelijkheid voor commerciële websites, webshops en apps in de hele EU, op basis van de WCAG-richtlijnen. Toegankelijkheid is daarmee een wettelijke eis en een gevraagde beroepscompetentie geworden, en in dit vak is het een rode draad: elk hoofdstuk behandelt de toegankelijkheidskant van zijn onderwerp. Voor dit hoofdstuk is dat: correcte semantiek, een kloppende hoofdingenstructuur, `lang` op het html-element, en straks het alt-attribuut.

Zoekmachines lezen je pagina op dezelfde manier: je title, je hoofdingen en je structuur bepalen mee hoe je gevonden wordt. En de jongste lezersgroep groeit het snelst: **AI-systemen en agents** die pagina's samenvatten, ernaar verwijzen of er taken op uitvoeren, doen dat betrouwbaarder naarmate de structuur betekenis draagt. Goede semantiek is dus geen beleefdheid; het is schrijven voor álle lezers van je pagina - en drie van de vier lezersgroepen zien alleen je boom.

1.9 Links: de essentie van het web

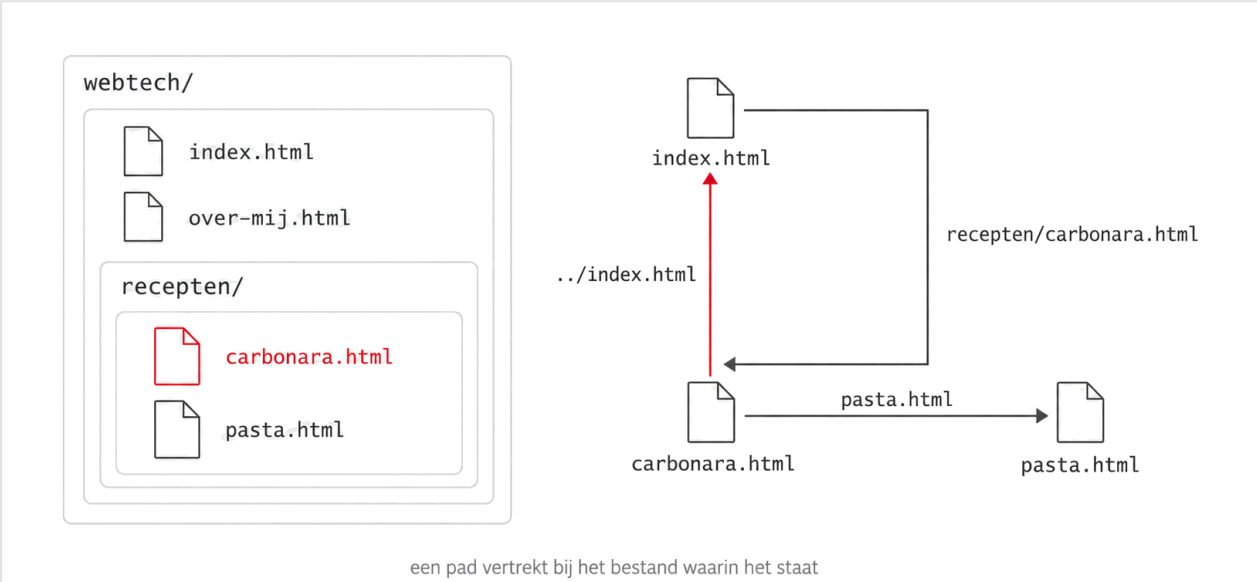
De "HT" in HTML en HTTP staat voor hypertext: tekst met sprongen erin. Het **a-element** (anchor) maakt die sprongen. Het verplichte `href`-attribuut bevat het adres van de bestemming; de inhoud tussen de tags is wat aanklikbaar wordt:

```
<a href="https://www.ap.be">Bezoek de site van AP</a>
```

Voor de waarde van `href` grijpen we terug naar de URL-anatomie van 1.2, want er zijn twee manieren om een adres te schrijven. Een **absolute URL** is het volledige adres, met scheme en host. Die gebruik je om te linken naar een andere website dan de jouwe. Absolute URL's kunnen lang en intimiderend ogen; laat je niet afschrikken, het blijft één attribuut met één waarde.

Een **relatieve URL** beschrijft de weg vanaf het huidige bestand, en die gebruik je voor alles binnen je eigen site. De regels zijn dezelfde als die van mappen op je computer. Een bestandsnaam alleen (`recepten.html`) wijst naar een bestand in dezelfde map. Een pad met mapnamen (`recepten/carbonara.html`) daalt af in submappen. En `../` klimt één map omhoog - herhaalbaar, dus `../../index.html` klimt er twee:

```
<a href="recepten.html">Alle recepten</a>
<a href="recepten/carbonara.html">Carbonara</a>
<a href="../index.html">Terug naar de homepagina</a>
```



Figuur 1.4 - Een relatief pad vertrekt bij het bestand waarin het staat: vanuit `carbonara.html` wijst `pasta.html` naar een buur en klimt `../index.html` één map omhoog; vanuit `index.html` daalt `recepten/carbonara.html` af.

Relatieve paden zijn de eerste plek waar het vak precisie afdwingt: één letter verschil, een vergeten map of een hoofdletter te veel, en de link is dood. Twee gewoontes voorkomen vrijwel alle ellende. Geef bestanden en mappen namen zonder spaties, zonder accenten en volledig in kleine letters (`mijn-recepten.html`, niet `Mijn Recepten.html`) - op jouw computer werkt het misschien allebei, op de server niet, en het verschil ontdek je pas na het publiceren. En denk vanaf het begin na over je mappenstructuur, want de paden schrijf je vanuit die structuur.

Een derde vorm zal je vaak tegenkomen in andermans code: paden die met `/` beginnen (`/recepten/carbonara.html`). Die starten vanaf de wortel van de site in plaats van vanaf het huidige bestand. Handig op grote sites, maar met een addertje: ze werken alleen op een echte server, en anders dan je zou verwachten wijst `/` op jouw GitHub Pages-adres niet naar jouw project maar naar het domein erboven. We gebruiken dit semester daarom consequent relatieve paden zonder beginnende slash; herken de vorm, gebruik hem nog niet.

1.10 Afbeeldingen

Het **img-element** plaatst een afbeelding in de pagina. Het is een void element (geen inhoud, geen sluittag) met twee verplichte attributen. **src** bevat het adres van het afbeeldingsbestand en werkt exact zoals href: absoluut of relatief, met dezelfde padregels - afbeeldingen krijgen in jouw projecten een eigen map:

```
<img src="afbeeldingen/carbonara.jpg" alt="Bord spaghetti carbonara met verse pecorino">
```

Het tweede verplichte attribuut verdient meer dan een vermelding. **alt** bevat het tekstalternatief voor de afbeelding: wat een schermlezer voorleest, wat de browser toont als het bestand niet laadt, en wat zoekmachines en AI-systemen lezen. Een goed alt-attribuut beschrijft wat de afbeelding *communiceert*, in de context van de pagina - niet dat het een afbeelding is (dat weet de lezer al). "Foto" is waardeloos; "bord spaghetti carbonara met verse pecorino" laat een blinde bezoeker hetzelfde weten als een ziende. Twee bijzondere gevallen: is de afbeelding puur decoratief en voegt ze geen informatie toe, schrijf dan een leeg alt-attribuut (`alt=""`) zodat schermlezers haar overslaan - het attribuut weglaten mag nooit. En bevat de afbeelding tekst (een logo met naam), dan is die tekst je alt.

Over de bestandsformaten hoef je dit semester maar dit te weten: **jpg** voor foto's, **png** voor afbeeldingen met scherpe randen of transparantie, **svg** voor logo's en iconen (het schaalt zonder kwaliteitsverlies, waarom hoor je in het motorkap-kader van hoofdstuk 3), en **webp** en **avif** als moderne, compactere vervangers voor jpg en png die je in professionele sites steeds vaker ziet. De browser toont ze allemaal met hetzelfde img-element.

Wil je een afbeelding een zichtbaar bijschrift geven, dan bestaat daar - geheel in de geest van dit hoofdstuk - een betekenisdragend duo voor: **figure** omhult de afbeelding, **figcaption** draagt het bijschrift:

```
<figure>
  <img src="afbeeldingen/carbonara.jpg" alt="Bord spaghetti carbonara">
  <figcaption>Het eindresultaat na twintig minuten werk.</figcaption>
</figure>
```

De width- en height-attributen die je in oudere code op img ziet, bespreken we pas in hoofdstuk 3 en 6: afmetingen zijn vormgeving, en vormgeving doen we in CSS. Eén uitzondering op die regel komt daar aan bod - ze heeft met laadgedrag te maken - maar tot dan schrijf je een img met src en alt, en niets meer.

1.11 Meer dan tekst en beeld: video, audio en iframe

Drie elementen maken de mediacatalogus compleet, en je zal ze sneller nodig hebben dan je denkt - de kans is groot dat je al in week twee een video op je persoonlijke site wil.

Het **video-element** speelt een videobestand af dat jij zelf serveert, precies zoals img een afbeeldingsbestand toont. Anders dan img heeft het wél een sluittag, en één attribuut is in de praktijk verplicht: **controls**, dat de afspeelknoppen van de browser toont. Zonder controls staat er een video die niemand kan starten:

```
<video src="media/demo.mp4" controls>
  <p>Je browser kan deze video niet afspelen.
     <a href="media/demo.mp4">Download het bestand.</a></p>
</video>
```

De inhoud tussen de tags is de reservetekst voor browsers die het bestand niet aankunnen - hetzelfde principe als alt bij img. Handige extra's: **poster** wijst naar een afbeelding die vóór het afspelen wordt getoond, en wie per se automatisch wil starten, moet weten dat browsers **autoplay** alleen toestaan in combinatie met **muted** - het web heeft collectief besloten dat pagina's die ongevraagd geluid maken een plaag zijn, en de browser handhaaft dat. Voor geluid zonder beeld bestaat het **audio-element**, dat identiek werkt: src, controls, sluittag, reservetekst. Als formaat volstaat dit semester mp4 voor video en mp3 voor audio; die spelen overal.

Maar eerlijk is eerlijk: de meeste video op het web serveer je niet zelf. Videobestanden zijn groot, en platforms als YouTube en Vimeo doen het hosten, comprimeren en streamen beter dan jouw GitHub Pages-site ooit zal doen. Daarvoor dient het derde element. Een **iframe** (inline frame) is een venster naar een ándere pagina, midden in de jouwe - letterlijk een pagina in een pagina. Kies je op YouTube voor *Delen* - *Insluiten*, dan krijg je zo'n iframe-snippet om te plakken:

```
<!-- ...replace VIDEO-ID with the id from the YouTube address... -->
<iframe src="https://www.youtube.com/embed/VIDEO-ID"
        title="Demo van het eindproject"
        allowfullscreen></iframe>
```

Twee dingen moet je daaraan zelf doen. Ten eerste het **title-attribuut**: een iframe is voor een schermlezer een gesloten doos, en title is het etiket erop - dezelfde rol als alt bij img, en net zo verplicht in dit vak. Ten tweede zal je merken dat een geplakte iframe een vast formaat heeft dat zich nergens iets van aantrekt; hoe je hem netjes laat meeschalen met zijn omgeving, is een layoutkwestie die in hoofdstuk 5 haar gereedschap krijgt (aspect-ratio). Tot dan mag hij er wat stijfjes bijstaan.

Eén waarschuwing tot slot: een iframe voert andermans pagina uit binnen de jouwe. Voor YouTube en soortgelijke gevestigde diensten is dat prima; sluit geen pagina's in van bronnen die je niet vertrouwt, om dezelfde reden dat je geen onbekende USB-stick in je laptop steekt.

1.12 Namen voor later: id en class

Tot slot twee attributen die elk HTML-element mag dragen en die je nu al moet herkennen, ook al bewijzen ze hun nut pas vanaf hoofdstuk 2: **id** en **class** geven een element een naam waarmee CSS en JavaScript het kunnen vastnemen.

Het verschil zit in uniciteit. Een **id** identificeert: de waarde mag op de hele pagina maar één keer voorkomen, zoals een studentennummer. Een **class** classificeert: dezelfde waarde mag op zoveel elementen staan als je wil, zoals een klasgroep, en één element mag meerdere classes dragen, gescheiden door spaties:

```
<main>
  <p class="intro">De eerste paragraaf, met een eigen rol.</p>
  <p>Een gewone paragraaf.</p>
  <p class="intro belangrijk">Deze paragraaf draagt twee classes.</p>
</main>
```

Kies betekenisvolle namen, in kleine letters en zonder spaties, en beschrijf de rol van het element, niet het gewenste uitzicht: `class="intro"` veroudert nooit, `class="rode-tekst"` wordt een leugen zodra het design verandert. Waarom je meestal naar class grijpt en maar zelden naar id, wordt duidelijk in hoofdstuk 2, wanneer we zien hoeveel gewicht een id in de schaal legt bij het geordende conflict van CSS.

1.13 Je pagina online

Daarmee is de cirkel van dit hoofdstuk rond: je weet wat een server is (1.2), je hebt een repository op GitHub (1.3), en je kan een correcte pagina schrijven (1.4 tot en met 1.12). De laatste stap is de drie samenbrengen, en die stap is verrassend klein.

Jouw repository `webtech` is gekoppeld aan **GitHub Pages**: telkens er iets verandert in je repository, plaatst GitHub de bestanden automatisch op zijn servers, en enkele ogenblikken later staat de nieuwe versie live op jouw adres - `https://jouwgebruikersnaam.github.io/webtech/` . De server zoekt daar, zoals elke webserver (1.2), naar een `index.html` ; de pagina's van labo 1 staan in de submap `labo-01/` , dus hun adres is `https://jouwgebruikersnaam.github.io/webtech/labo-01/index.html` . In labo 1 breng je je bestanden in de repository door je map in de browser te uploaden. Vanaf labo 2 wordt dat de vaste werkstroom van elke week in dit vak: bewerk je bestanden in VS Code, bekijk het resultaat live met Live Server, en als je tevreden bent: *Commit & Sync* met een korte boodschap die zegt wat je deed. De stappenkaart met schermafbeeldingen staat in appendix A; na labo 2 heb je haar niet meer nodig.

Eén gewoonte geven we je vanaf dag één mee, omdat ze het verschil maakt tussen een geschiedenis die je helpt en een die niets zegt: commit per afgewerkte stap, niet per week. "Navigatie toegevoegd aan alle pagina's" is een commit; "van alles" is er geen. Je geschiedenis

wordt zo een logboek van je werk - voor jezelf, voor je docenten, en voor de feedback-tooling die vanaf labo 2 met elke wijziging meekijkt.

Het visitekaartje van je pagina

Nu je pagina een publiek adres heeft, verdient ze ook een visitekaartje: drie regels in de head die niets aan de pagina zelf veranderen, maar alles aan hoe ze de buitenwereld tegemoet treedt. De **favicon** is het pictogrammetje in het browsertabblad: één regel (`<link rel="icon" href="favicon.svg">`) en jouw site is geen naamloos tabblad meer tussen de twintig andere. De **meta description** (`<meta name="description" content="...">`) is de samenvatting van één of twee zinnen die zoekmachines onder je title tonen in de resultatenlijst; schrijf je haar niet, dan knipt de zoekmachine zelf een willekeurig fragment uit je pagina. En de **Open Graph-tags** bepalen hoe je pagina eruitziet wanneer iemand de link deelt in een chat of op sociale media - het voorbeeldkaartje met titel, omschrijving en afbeelding:

```
<!-- ...inside head... -->
<meta property="og:title" content="Portfolio van Tim Peeters">
<meta property="og:description" content="Student Toegepaste Informatica aan AP.">
<meta property="og:image"
content="https://timpeeters.github.io/webtech/afbeeldingen/kaart.png">
```

Let bij og:image op het absolute adres: het kaartje wordt opgebouwd door de server van de chat-app, niet door jouw pagina, dus een relatief pad wijst voor hém nergens naartoe. Voor je persoonlijke site is dit drietal geen verplichting van week één, maar wel de afwerking die het verschil maakt zodra je je adres begint rond te sturen - en rondsturen is precies de bedoeling.

De validator

Eén instrument nog, en het is er één voor je hele loopbaan. Je leest in het motorkap-kader hieronder dat foute HTML niet luid faalt maar stil: de browser herstelt zwijgend, en jij ziet niets. Er bestaat gelukkig een lezer die wél luid faalt: de **HTML-validator** van het W3C (validator.w3.org). Plak er het adres van je gepubliceerde pagina in en je krijgt een lijst van alles wat formeel niet klopt, met regelnummers. Maak er een gewoonte van om elke pagina die je publiceert één keer door de validator te halen - het is de snelste manier om fouten te vinden die je oog nooit zou zien, en in de review-oefeningen van deze syllabus is het een vast onderdeel van de verificatiestap.

Onder de motorkap: van adresbalk tot pixels

Dit kader gaat dieper dan het hoorcollege. Je hebt het niet nodig om de oefeningen te maken - wel om te begrijpen waarom de regels van dit hoofdstuk bestaan.

Wat gebeurt er precies tussen het moment waarop je op enter drukt en het moment waarop de pagina er staat? De reis heeft vijf etappes.

Opzoeken. De browser kent van `www.ap.be` alleen de naam, geen locatie. Via het **DNS** (Domain Name System, een wereldwijd gedistribueerd adresboek) vertaalt hij de naam naar een IP-adres: het nummer van de machine die de site serveert.

Ophalen. Nu volgt het HTTP-gesprek uit 1.2: verzoek, antwoord, statuscode, en als alles goed gaat een stroom bytes die jouw HTML-bestand vormen. Bij HTTPS gaat daar nog een korte handdruk aan vooraf, waarin de server zijn certificaat toont en beide kanten een sleutel afspreken waarmee de rest van het gesprek versleuteld wordt.

Parsen. De browser leest die bytes van boven naar beneden en hakt ze in stukjes: openingstag, tekst, sluittag. Dit heet **parsen**, en het resultaat is de boom uit 1.6 - de DOM. Vanaf hier bestaat jouw bestand voor de browser niet meer; alles wat volgt, werkt op de boom. Dat is de diepere reden waarom de DevTools je de boom tonen en niet je bestand: je kijkt naar wat de browser daadwerkelijk heeft.

Aankleden en meten. Op de boom past de browser vervolgens de stijlregels toe (van jou, of zijn eigen standaardstijlen - dat verhaal begint in hoofdstuk 2) en rekent hij voor elke knoop uit waar hij komt te staan en hoe groot hij wordt (het boxmodel van hoofdstuk 3).

Schilderen. Ten slotte zet de browser het resultaat om in pixels op je scherm. Elke keer dat er iets verandert - jij past CSS aan, JavaScript wijzigt de boom - doorloopt hij de laatste etappes opnieuw. Hoe je die herberekeningen bewust gebruikt (en soms bewust vermijdt), komt terug bij transities en animaties in hoofdstuk 8.

Er zit in de parse-etappe nog een venijnig detail dat elke beginner een keer in de val lokt. HTML-parsers zijn radicaal vergevingsgezind: bij fouten stopt de browser niet met een foutmelding, maar *herstelt* hij je document volgens vaste regels tot een geldige boom. Vergeet je een sluittag, kruis je twee elementen, zet je een element op een plek waar het niet mag staan - de browser bouwt zwijgend de boom die volgens de herstelregels het dichtst in de buurt komt. Dat klinkt vriendelijk, maar het betekent: **foute HTML faalt niet luid, ze faalt stil.** De pagina "werkt", alleen is de boom niet wat jij dacht - en over drie weken snap je niet waarom een CSS-regel het ene lijstitem wel raakt en het andere niet. Daarom zijn de regels van dit hoofdstuk (correcte nesting, li's direct in ul, elk stuk informatie in een element) geen etiquette, en daarom is de Elements-tab je waarheidsinstrument: daar zie je de boom ná herstel. Wijkt hij af van wat je schreef, dan heeft de parser iets voor je "opgelost".

Veelgemaakte fouten

Dit katern keert in elk hoofdstuk terug: de fouten die we elk jaar zien, met symptoom, oorzaak en remedie. De debug-oefeningen en de feedback op je labo's putten er rechtstreeks uit en verwijzen naar deze nummers.

F1.1 - Naakte tekst. *Symptoom*: tekst staat op de pagina maar gedraagt zich vreemd zodra je gaat stylen. *Oorzaak*: de tekst staat rechtstreeks in body of main, zonder omhullend element. *Remedie*: elk stuk informatie krijgt een element dat zegt wat het is; lopende tekst hoort in een p.

F1.2 - Hoofdingen op uitzicht gekozen. *Symptoom*: een h4 pal onder een h1, want "dat was de juiste grootte". *Oorzaak*: hoofdingen behandeld als lettergroottes in plaats van als inhoudsopgave. *Remedie*: niveaus volgen de structuur (geen niveaus overslaan, één h1); groottes regel je in hoofdstuk 2 met CSS.

F1.3 - Gekruiste tags. *Symptoom*: styling of gedrag "lekt" naar tekst waar het niet hoort. *Oorzaak*: `<p><em>tekst</p></em>` - elementen overlappen in plaats van te nesten; de diepere oorzaak is dat er niet in bomen gedacht is. *Remedie*: een element zit volledig in zijn ouder; controleer de herstelde boom in de Elements-tab.

F1.4 - Andere kinderen dan li in een lijst. *Symptoom*: bullets op rare plaatsen, items die niet uitlijnen. *Oorzaak*: tekst of elementen rechtstreeks in ul of ol geplaatst. *Remedie*: ul en ol hebben uitsluitend li-kinderen; al de rest zit ín een li - ook geneste lijsten.

F1.5 - Vergeten of nietszeggende alt. *Symptoom*: geen zichtbaar symptoom, en dat is precies het probleem. *Oorzaak*: alt weggelaten of ingevuld als "foto" of "afbeelding1". *Remedie*: beschrijf wat de afbeelding communiceert; decoratief beeld krijgt alt="".

F1.6 - Spaties en hoofdletters in bestandsnamen. *Symptoom*: alles werkt lokaal, maar na publicatie zijn links dood en afbeeldingen kapot. *Oorzaak*: `Mijn Foto.JPG` - jouw besturingssysteem is soepel, de server niet. *Remedie*: kleine letters, koppeltekens, geen spaties of accenten - vanaf het eerste bestand.

F1.7 - Verdwaalde relatieve paden. *Symptoom*: een link of afbeelding werkt op de ene pagina wel en op de andere niet. *Oorzaak*: het pad is geschreven vanuit de verkeerde map; relatieve paden vertrekken bij het bestand waarín ze staan. *Remedie*: teken je mappenstructuur en wandel het pad na, map voor map, vanaf het bestand zelf.

F1.8 - Structuurelementen in de verkeerde rol. *Symptoom*: sitenavigatie in main, of paginatitel in de footer. *Oorzaak*: de vraag "keert dit terug op elke pagina of is dit uniek voor deze pagina?" is niet gesteld. *Remedie*: stel exact die vraag per blok; terugkerend hoort in header of footer, uniek in main.

F1.9 - Div-soep. *Symptoom*: een technisch werkende pagina die volledig uit divs bestaat. *Oorzaak*: de neutrale container gebruikt zonder eerst naar een betekenisdragend element te zoeken - een fout die je ook in veel gegenereerde code ziet. *Remedie*: de rangorde van 1.8: eerst semantiek, div als laatste redmiddel.

F1.10 - De boom niet gecontroleerd. *Symptoom*: "het ziet er goed uit, dus het is goed." *Oorzaak*: alleen naar het gerenderde resultaat gekeken, nooit naar de structuur. *Remedie*: maak de Elements-tab en de validator een vast onderdeel van je afwerkroutine; het motorkap-kader legt uit waarom het oog hier niet volstaat.

F1.11 - Media zonder etiket of bediening. *Symptoom*: een video die niemand kan starten, of een iframe dat voor schermlezers een naamloos gat in de pagina is. *Oorzaak*: controls vergeten op video of audio, title vergeten op iframe. *Remedie*: video en audio krijgen controls, elk iframe krijgt een beschrijvende title - dezelfde reflex als alt bij img.

Met AI aan je zij: gegenereerde HTML lezen

Vraag een taalmodel om een webpagina en je krijgt er binnen seconden een die werkt. In dit terugkerende kader leer je per hoofdstuk waar je dan op moet letten; bij HTML zijn dat deze vier patronen.

Div-soep en overbodige lagen. Modellen zijn getraind op miljoenen bestaande pagina's, en het gemiddelde van het web is geen goed voorbeeld. Je krijgt vaak stapels geneste divs met classes als `container`, `wrapper` en `content` waar één semantisch element volstond, en `<div class="header">` waar `<header>` bestaat. De structuur wérkt, maar drie van je vier lezersgroepen (1.8) verliezen erbij. Jouw controlevraag bij elke gegenereerde container: welk betekenisdragend element had hier gekund?

Plausibele maar lege alt-teksten. Gegenereerde afbeeldingen krijgen keurig een alt-attribuut - modellen "weten" dat het moet - maar de inhoud is vaak generiek ("een afbeelding van het product") of gokt naar wat het model niet kan zien. Formeel correct, functioneel waardeloos. Alt-teksten schrijven blijft mensenwerk: jij weet wat de afbeelding in déze context communiceert.

Verouderd idioom. Trainingsdata bevat twintig jaar web, dus gegenereerde code bevat geregeld het web van tien jaar geleden: `<b>` en `<i>` waar strong en em horen, `<br>`-tags als witruimte-gereedschap, inline styles in een style-attribuut (daarover meer in hoofdstuk 2). Het werkt allemaal; het is allemaal een teken dat je de output niet blind moet overnemen.

Hoofdingen op uitzicht. Ook modellen maken fout F1.2: ze kiezen wel eens hoofdingniveaus op verwachte grootte in plaats van op structuur. Jouw inhoudsopgave-test (alleen de hoofdingen lezen) werkt op gegenereerde pagina's net zo goed als op je eigen werk.

De rode draad, hier en in elk volgend hoofdstuk: het model produceert, jij beoordeelt - en beoordelen kan alleen wie de regels zelf kent. In de oefeningen hieronder doe je dit voor het eerst systematisch.

Oefeningen

Labo 1 bevat de opgaven van deze week, met de precieze bestandsnamen en de inleverinstructies. Hieronder staat wat je na dit hoofdstuk moet kunnen, zodat je jezelf kan testen. De AI-labels staan in 1.1.

Drills (zonder AI aanbevolen)

- **D1.1** Schrijf het volledige skelet van een HTML-document uit het hoofd, inclusief lang, charset, viewport en title. Dit moet je op het einde van deze week foutloos kunnen, zonder na te denken.
- **D1.2** Je krijgt een platte tekst (een recept: titel, inleiding, ingrediënten, stappen, bronvermelding). Zet hem om in correct gestructureerde HTML met hoofdingen, paragrafen en de juiste lijstsoorten.

- **D1.3** Teken op papier het boomdiagram van een gegeven HTML-fragment, met de begrippen parent, child en sibling. Controleer jezelf daarna in de Elements-tab.
- **D1.4** Bouw een geneste lijst van drie niveaus diep (bv. opleidingen, met per opleiding vakken, met per vak onderwerpen) zonder één nesting-fout.
- **D1.5** Bouw een minisite van drie pagina's (index, recepten in een submap, over mij) met een gedeelde navigatie en werkende relatieve links in beide richtingen, inclusief een `../`-pad.
- **D1.6** Voeg aan een gegeven pagina drie afbeeldingen toe vanuit een afbeeldingenmap: één informatieve (met goede alt), één decoratieve (met leeg alt) en één met bijschrift via figure en figcaption.
- **D1.7** Sluit een YouTube-video van je keuze in via een iframe met een beschrijvende title, en voeg daarnaast een eigen (kort) videobestand toe met controls en reservetekst. Haal daarna de gepubliceerde pagina door de W3C-validator en los alles op wat hij meldt.

Debug en review (zonder AI)

- **R1.1** Je krijgt een pagina met tien fouten uit het foutenkatern. Vind ze alle tien, benoem per fout wát er mis is en waaróm het een probleem is (ook als de pagina "werkt"), en verbeter ze. Gebruik de Elements-tab om je diagnoses te staven.
- **R1.2** Je krijgt twee versies van dezelfde pagina: identiek gerenderd resultaat, verschillende markup. Beargumenteer in vijf zinnen welke versie beter is, met de vier lezersgroepen uit 1.8 als toetssteen.

Bouwen en verantwoorden (met AI of zonder AI, jouw keuze, mét verantwoording)

- **B1.1** De pagina's die je in labo 1 online zet, zijn versie één van je persoonlijke site: `index.html` met het grondplan uit 1.8 (header met navigatie, main, footer), een voorstelling van jezelf, lijsten, minstens één afbeelding met goede alt en één externe link. Deze site groeit elke week mee met het vak; dit is je fundament, dus bouw hem zoals iemand die weet dat er twaalf verdiepingen bovenop komen. Labo 1 is zonder AI. Kies je daarna, bij het uitbreiden van je site, voor de AI-route, lever dan naast de site ook in: je prompt, de onbewerkte eerste output, en een lijst van wat je verbeterde en waarom - beoordeeld wordt de kwaliteit van je kritiek.

Verder lezen

- MDN, *How the web works*: https://developer.mozilla.org/en-US/docs/Learn_web_development/Getting_started/Web_standards/How_the_web_works
- MDN, *Getting started with HTML*: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Structuring_content/Basic_HTML_syntax
- MDN, *HTML text fundamentals*: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Structuring_content/Headings_and_paragraphs
- MDN, *Document and website structure*: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Structuring_content/Structuring_documents

- MDN, *Images in HTML*: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Structuring_content/HTML_images